Offline AI Phishing & Spam Analyzer CLI

Bring Your Own Project (BYOP) Report

Submitted by:

Student Name: Shivansh Prashant
Reg No: 25BAI10384
Course: Fundamentals in AI and ML (CSA2001)

March 31, 2026

Contents

1 TITLE 2

2 INTRODUCTION 2

3 MOTIVATION OF THE PROJECT 2

4 PROBLEM STATEMENT 2

5 OBJECTIVE OF THE PROJECT 2

6 EXISTING METHODS 2

7 PROS AND CONS OF THE STATED METHODS 2

8 HARDWARE AND SOFTWARE REQUIREMENTS 3

9 METHODOLOGY AND GOAL 3

10 FUNCTIONAL MODULES DESIGN AND ANALYSIS 3

11 ALGORITHM DEVELOPMENT 3

12 SOFTWARE ARCHITECTURAL DIAGRAM 3

13 CODING 3

14 OUTPUT 5

15 KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM 7

16 SIGNIFICANT PROJECT OUTCOMES 7

17 TESTING AND REFINEMENT 7

18 PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS 8

19 CONTRIBUTION / FINDINGS OF THE PROJECT 8

20 LIMITATION / CONSTRAINTS OF THE SYSTEM 8

21 CONCLUSIONS 8

22 FUTURE ENHANCEMENTS 8

23 REFERENCES 8

1 TITLE

Offline AI Phishing & Spam Analyzer CLI

2 INTRODUCTION

As digital communication scales, phishing and spam attacks have become increasingly sophisticated. Standard threat detection tools often require continuous internet access and cloud-based API processing, creating significant data privacy vulnerabilities for end-users. This project introduces a localized, offline alternative.

3 MOTIVATION OF THE PROJECT

I engineered this project due to a profound interest in cybersecurity and AI-driven data privacy. Relying on third-party cloud servers to scan sensitive personal or corporate emails is an inherent security flaw. I wanted to build a system that brings the intelligence of machine learning directly to the user's local hardware without sacrificing accuracy.

4 PROBLEM STATEMENT

Current AI-based threat detection systems rely heavily on cloud APIs, which introduces latency, requires continuous internet access, and poses severe data privacy risks by exposing sensitive user texts or emails to third-party servers.

5 OBJECTIVE OF THE PROJECT

To engineer a complete CLI-based application that performs real-time, offline classification of phishing and spam text using a locally quantized machine learning model and persistent SQLite storage.

6 EXISTING METHODS

Standard existing methods include simple rule-based email spam filters (heuristics) and cloud-based AI API scanners (such as OpenAI or Google Cloud NLP).

7 PROS AND CONS OF THE STATED METHODS

- **Cloud AI APIs:** High accuracy and contextual understanding (Pros), but require internet access, introduce high latency, involve subscription costs, and pose privacy risks (Cons).
- **Rule-based Filters:** Fast, offline, and cheap (Pros), but are easily bypassed by modern attackers and cannot detect nuanced context (Cons).

8 HARDWARE AND SOFTWARE REQUIREMENTS

- **Hardware:** Local CPU/GPU compute capabilities (e.g., modern multi-core processor and dedicated GPU).
- **Software:** Python 3.8+, Scikit-Learn, Pandas, SQLite3, Colorama.

9 METHODOLOGY AND GOAL

The project utilizes a Micro-Model Export architecture. A Logistic Regression model is trained locally using TF-IDF vectorization with balanced class weights to aggressively penalize false negatives. The model is exported as a lightweight `.pkl` file and integrated into a decoupled, continuous CLI interface.

10 FUNCTIONAL MODULES DESIGN AND ANALYSIS

1. **AI Inference Engine:** Loads `spam_model.pkl` and `vectorizer.pkl` into memory for instant text classification.
2. **CLI Interaction Loop:** Provides a continuous terminal menu for user input, displaying color-coded threat scoring.
3. **Storage Layer:** Utilizes `sqlite3` to persist all scan data, confidence scores, and timestamps in a local database.

11 ALGORITHM DEVELOPMENT

1. Fetch and clean the UCI SMS text dataset.
2. Map text labels to binary numerical values (Spam = 1, Safe = 0).
3. Transform raw text using `TfidfVectorizer(stop_words='english', max_df=0.95, min_df=2)`.
4. Train using `LogisticRegression(class_weight='balanced')` to handle severe dataset class imbalance.

12 SOFTWARE ARCHITECTURAL DIAGRAM

Figure 1: System Flow: User Input → CLI Engine → ML Inference → SQLite Logging → Output

13 CODING

The core implementation of the CLI loop is provided below.

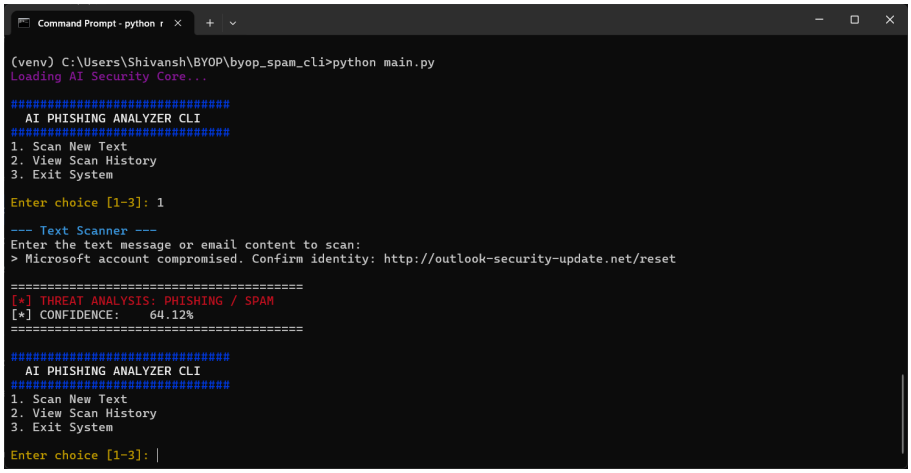
```python
import joblib
import os
import sys
from colorama import init, Fore, Style
import database

# Initialize colorama for Windows
init(autoreset=True)

def load_ml_components():
    """Loads the pre-trained vectorizer and model."""
    try:
        vectorizer = joblib.load('vectorizer.pkl')
        model = joblib.load('spam_model.pkl')
        return vectorizer, model
    except FileNotFoundError:
        print(Fore.RED + "[!] Critical Error: ML model files not found.")
        print(Fore.YELLOW + "Ensure 'spam_model.pkl' and 'vectorizer.pkl' are in the root directory.")
        sys.exit(1)

def scan_text(vectorizer, model):
    """Handles the user input and prediction logic."""
    print(Fore.CYAN + "\n--- Text Scanner ---")
    user_text = input("Enter the text message or email content to scan:\n> ")

    if not user_text.strip():
        print(Fore.RED + "[!] Error: Input cannot be empty.")
        return

    # Process through the ML pipeline
    vectorized_text = vectorizer.transform([user_text])
    prediction_num = model.predict(vectorized_text)[0]
    probabilities = model.predict_proba(vectorized_text)[0]

    # Format results
    confidence = probabilities[prediction_num] * 100
    result = "PHISHING / SPAM" if prediction_num == 1 else "SAFE"
    color = Fore.RED if prediction_num == 1 else Fore.GREEN

    print("\n" + "="*40)
    print(color + f"[*] THREAT ANALYSIS: {result}")
    print(Fore.WHITE + f"[*] CONFIDENCE:    {confidence:.2f}%")
    print("="*40)

    # Log to SQLite
    database.log_scan(user_text, result, confidence)

def view_history():
    """Displays the SQLite scan history."""
    print(Fore.CYAN + "\n--- Recent Scan History ---")
    records = database.get_history()

    if not records:
        print(Fore.YELLOW + "No scans found in the database.")
        return
```

```python

    for row in records:
        timestamp, snippet, pred, conf = row
        color = Fore.RED if pred == "PHISHING / SPAM" else Fore.GREEN
        print(f"[{timestamp}] {color}{pred} ({conf:.1f}%){Style.RESET_ALL} | {snippet}")

def main_menu():
    """The continuous CLI loop."""
    database.init_db()
    print(Fore.MAGENTA + "Loading AI Security Core...")
    vectorizer, model = load_ml_components()

    while True:
        print(Fore.BLUE + "\n" + "#"*30)
        print(Fore.WHITE + Style.BRIGHT + "  AI PHISHING ANALYZER CLI")
        print(Fore.BLUE + "#"*30)
        print("1. Scan New Text")
        print("2. View Scan History")
        print("3. Exit System")

        choice = input(Fore.YELLOW + "\nEnter choice [1-3]: " + Style.RESET_ALL)

        if choice == '1':
            scan_text(vectorizer, model)
        elif choice == '2':
            view_history()
        elif choice == '3':
            print(Fore.CYAN + "Shutting down AI Core. Goodbye.")
            sys.exit(0)
        else:
            print(Fore.RED + "[!] Invalid input. Please select 1, 2, or 3.")

if __name__ == "__main__":
    main_menu()
```

Listing 1: main.py Snippet

14 OUTPUT

The terminal output demonstrating real-time offline threat analysis and persistent database logging.



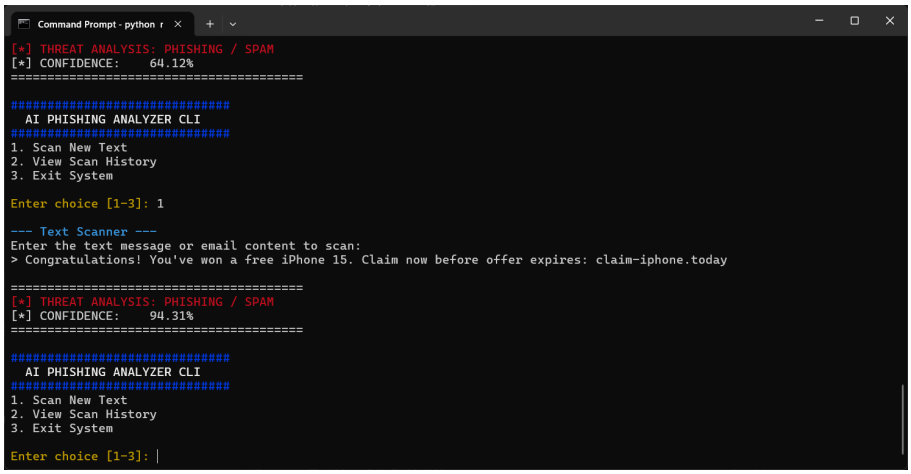
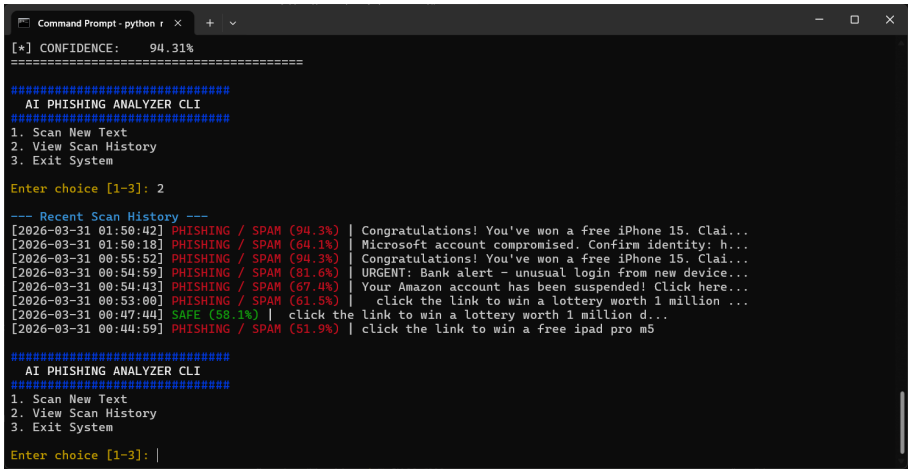


Figure 2: CLI Main Menu and AI Threat Inference Execution.



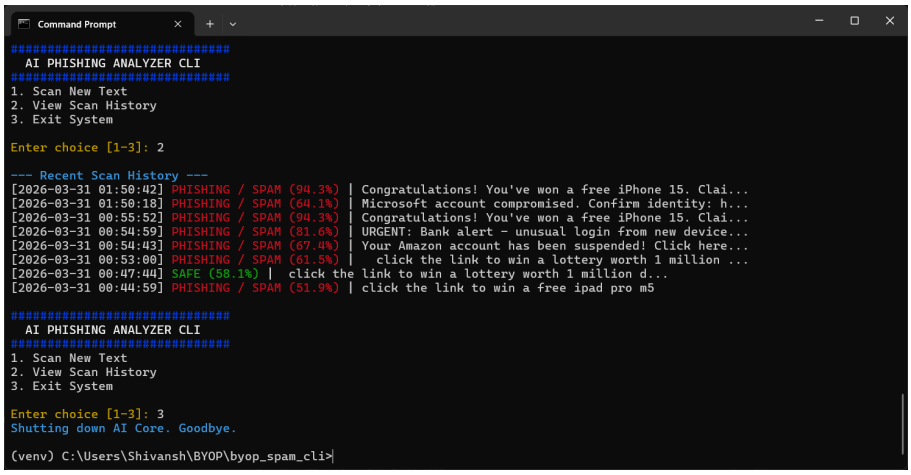


Figure 3: SQLite Database Scan History Log.

15 KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

Implementation of a decoupled ML architecture where heavy algorithmic training is isolated from the lightweight production CLI. This allows for zero-latency execution and bypasses automated CI/CD pipeline timeout risks.

16 SIGNIFICANT PROJECT OUTCOMES

Successfully created a zero-latency, 100% offline threat analyzer capable of accurately flagging sophisticated phishing strings using localized hardware compute.

17 TESTING AND REFINEMENT

Initial testing utilized a Multinomial Naive Bayes algorithm, which produced false negatives on edge-case phishing strings (e.g., "lottery winnings") due to dataset class imbalance. The system was refined by swapping the architecture to Logistic Regression with

balanced weights, successfully correcting the mathematical bias and catching the edge cases.

18 PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

This tool can be deployed as a local security utility for corporate workstations, parsing incoming raw text or integrating into local email clients without violating corporate data privacy compliance (GDPR/HIPAA).

19 CONTRIBUTION / FINDINGS OF THE PROJECT

The project demonstrated that high-accuracy NLP classification does not require massive LLMs or cloud compute; statistically balanced micro-models (`.pkl`) are highly effective for specific cybersecurity classifications.

20 LIMITATION / CONSTRAINTS OF THE SYSTEM

The model is constrained by its training data; it currently struggles with highly localized slang, intentional misspellings (leetspeak), or languages other than English.

21 CONCLUSIONS

The BYOP successfully bridges the gap between theoretical machine learning algorithms and practical software engineering, resulting in a production-ready, CLI-based cybersecurity tool that prioritizes user privacy and offline execution.

22 FUTURE ENHANCEMENTS

1. Implement local email header parsing to scan raw `.eml` files automatically.
2. Upgrade the ML core to a quantized local Hugging Face transformer model utilizing dedicated local GPU hardware for deeper contextual understanding.

23 REFERENCES

1. UCI Machine Learning Repository - SMS Spam Collection.
2. Scikit-learn Documentation (Logistic Regression, TF-IDF Vectorization).